

FECAP — Fundação Escola de Comércio Álvares Penteado

Ciência da Computação — Noturno — 5º Semestre

Inteligência Artificial e Aprendizado de Máquina

Projeto Lideranças Empáticas

Identificação e Contabilização de Produtos por Visão Computacional

Documentação técnica do modelo de IA — Entrega 2

Integrantes do grupo

Felipe Vallim Soares — 24026060

Guilhermy Mariano Lisboa Garcia — 23025371

Gustavo Oliveira Demetrio — 24026213

Saulo Pereira de Jesus — 24026095

São Paulo, 2026

1. Introdução

O projeto Lideranças Empáticas é uma iniciativa que une impacto social e educação empreendedora por meio da arrecadação de alimentos e recursos. Uma das dificuldades operacionais é realizar a contagem confiável da arrecadação por equipe e por tipo de alimento.

Para resolver esse problema, esta entrega apresenta um modelo de Inteligência Artificial baseado em Visão Computacional, capaz de identificar visualmente cinco produtos diferentes (de modo fixo ou em movimento) e contabilizar automaticamente quantidade, peso estimado e valor monetário de cada item detectado.

O modelo foi treinado, avaliado e disponibilizado em um Jupyter Notebook que cobre todo o pipeline: geração do dataset, augmentation, treinamento da rede e validação das métricas. A integração com o sistema operacional do projeto (câmera ao vivo, banco de dados e armazenamento de evidências) foi implementada nos módulos da pasta *camera-ai*.

2. Produtos Identificados pelo Modelo

O modelo foi treinado para reconhecer cinco categorias de pacotes de alimentos comumente arrecadados. Para cada classe, foram fixados valores padrão de peso e preço, usados no cálculo automático do valor da arrecadação:

Produto	Peso (g)	Preço/kg (R\$)	Valor unit. (R\$)
Arroz	1.000	5,50	5,50
Feijão	1.000	7,50	7,50
Açúcar	1.000	4,50	4,50
Café	500	50,00	25,00
Macarrão	500	8,00	4,00

Os pesos e preços ficam centralizados no arquivo *camera-ai/config.py* (constantes `CATEGORY_WEIGHTS_G` e `CATEGORY_PRICES_BRL_PER_KG`), o que permite ajustes rápidos sem alterar a lógica do código.

3. Tecnologia e Modelo Utilizado

A solução foi construída em Python, usando as seguintes bibliotecas e ferramentas:

- Ultralytics YOLOv8 — modelo de detecção de objetos (versão nano, ~6 MB);
- OpenCV — captura da câmera, pré-processamento e anotação visual dos frames;
- NumPy — manipulação de imagens em formato matricial;
- SQLAlchemy + PostgreSQL — persistência das detecções no banco de dados;
- Boto3 — envio assíncrono dos frames de evidência para o Amazon S3;
- Jupyter Notebook — ambiente de treinamento e validação do modelo.

Optamos pela família YOLO por entregar detecção e classificação em uma única passagem, com baixa latência mesmo em CPU — o que atende o requisito de quase tempo real previsto no projeto.

3.1 Dataset e Aumentação de Dados

Como o conjunto inicial de fotos era pequeno, aplicamos um pipeline de data augmentation no notebook `food_classifier.ipynb` que gera dez variações sintéticas para cada imagem original (rotação $\pm 20^\circ$, espelhamento horizontal/vertical, ajuste de brilho/contraste, blur gaussiano, ruído gaussiano e zoom $\pm 15\%$). O resultado final foi:

Classe	Imagens base	Após augmentation
Arroz	17	187
Feijão	28	308
Açúcar	16	176
Café	9	99
Macarrão	18	198
Total	88	968

O conjunto foi separado em 80% para treino e 20% para validação, totalizando aproximadamente 774 imagens de treino e 194 de validação.

3.2 Hiperparâmetros do Treinamento

- Modelo base: YOLOv8n (nano), pré-treinado no dataset COCO;
- Épocas: 30 | Batch size: 8 | Tamanho de imagem: 640 × 640;
- Otimizador: AdamW (selecionado automaticamente);
- Dispositivo: CPU | AMP (mixed precision) ativado.

3.3 Métricas de Avaliação

As métricas abaixo foram extraídas do peso best.pt — a melhor época do treinamento, validada sobre o conjunto de validação:

Métrica	Valor
mAP50	99,5%
mAP50-95	99,5%
Precision	99,7%
Recall	99,9%

Os resultados são consistentes com um cenário de operação controlado (fundo escuro, iluminação fixa) e indicam que o modelo distingue muito bem as cinco classes nas condições previstas pelo projeto.

4. Como Funciona a Contagem

A contagem é feita em tempo real, frame a frame, por uma sequência de três componentes que evitam dupla contagem do mesmo pacote:

1. Detecção (ml/inference.py): o YOLOv8 analisa cada frame e devolve as caixas dos itens com confiança $\geq 0,75$.
2. Tracking (tracking/tracker.py): um CentroidTracker associa as detecções entre frames calculando a distância euclidiana entre centroides (limite de 100 px, item descartado após 5 frames sem aparecer).
3. Estabilidade: cada track precisa manter a mesma classe por 10 frames consecutivos antes de ser considerado estável — isso filtra detecções incertas.
4. Linha virtual de contagem (tracking/line_counter.py): uma linha horizontal posicionada a 50% da altura do frame; quando o centroide de um track estável cruza a linha, o item é contabilizado uma única vez (o ID do track previne dupla contagem).

5. Persistência (services/detection_writer.py): cada contagem confirmada é gravada na tabela ai_detections do PostgreSQL, com timestamp, equipe, confiança, peso e valor.
6. Evidência (services/s3_uploader.py): o frame anotado é enviado em background para um bucket S3, e a chave do arquivo é vinculada ao registro no banco.

5. Saída no Terminal e Registros no Backend

Cada vez que um item é confirmado, o serviço de captura imprime no terminal uma linha resumindo a detecção. Exemplo real do log:

```
[CONTADO] arroz (arroz) conf=79% peso=1000.0g preco=R$5.50 | total=1
```

Ao encerrar a sessão, o sistema apresenta o resumo total por categoria:

```
[INFO] Sessao encerrada. Total detectado: 1  
arroz: 1  
feijao: 0  
outros: 0
```

Em paralelo, cada detecção gera uma linha na tabela ai_detections com os seguintes campos: id (UUID), item_name, category (arroz / feijão / outros), estimated_weight_g, estimated_price_brl, confidence, detected_at (timestamp), team_id (equipe responsável), operator_name e s3_key (chave da imagem-evidência no S3).

Esses registros alimentam diretamente os relatórios por equipe, por categoria e por período exigidos no requisito de backend do projeto.

6. Demonstração Visual do Modelo em Operação

A imagem abaixo mostra o modelo em execução sobre uma webcam capturando um pacote real de arroz. A interface visual sobreposta ao vídeo é gerada pelo módulo *capture_service.py*:



```
[CONTADO] arroz (arroz) conf=79% peso=1000.0g preco=R$5.50 | total=1  
[INFO] Encerrando...
```

```
[INFO] Sessao encerrada. Total detectado: 1  
arroz: 1  
feijao: 0  
outros: 0
```

Pontos importantes da demonstração:

- Contadores no canto superior esquerdo: total de itens detectados na sessão e separação por categoria (arroz, feijão, outros).
- Linha amarela horizontal (rótulo "CONTAGEM"): linha virtual de cruzamento. O item só é contabilizado quando seu centro cruza essa linha, garantindo contagem única.
- Painel "Histórico de detecções" (canto superior direito): lista os últimos itens contabilizados com produto, peso e valor unitário, além do total acumulado da sessão (no exemplo: 1000 g e R\$ 5,50).

- Logs no terminal (parte inferior da imagem): cada item contabilizado emite uma linha [CONTADO] com classe detectada, confiança, peso e preço; ao encerrar, [INFO] Sessão encerrada exibe o resumo final por categoria.

Esse mesmo conjunto de informações (peso, valor, classe e timestamp) é o que é gravado no banco de dados PostgreSQL e enviado como evidência ao S3 — fechando o ciclo entre detecção, contabilização e auditoria.

7. Conclusão

O modelo entregue cumpre integralmente os requisitos definidos para esta etapa do projeto: identificação visual de cinco produtos distintos (arroz, feijão, açúcar, café e macarrão), detecção tanto em modo fixo quanto em movimento, e contabilização automática de quantidade, peso e valor da arrecadação.

A escolha do YOLOv8n trouxe um equilíbrio adequado entre acurácia (mAP50 de 99,5% no conjunto de validação) e custo computacional, permitindo execução fluida em CPU. O pipeline completo — detecção, tracking por centroide, linha virtual de contagem e persistência no banco — garante que cada pacote seja contado uma única vez, e que cada contagem fique registrada com sua respectiva evidência visual.

Os artefatos entregues nesta etapa são: o notebook *food_classifier.ipynb* (geração do dataset, augmentation, treino e avaliação), o peso treinado e os módulos da pasta *camera-ai em backend* responsáveis pela inferência em tempo real, contagem e integração com o projeto em si.